NHÓM 3

Tiếng Nói Từ Bàn Tay

Ứng dụng công nghệ Deep Learning để chuyển đổi ngôn ngữ hình thể sang tiếng Việt Nam

Made with GAMMA

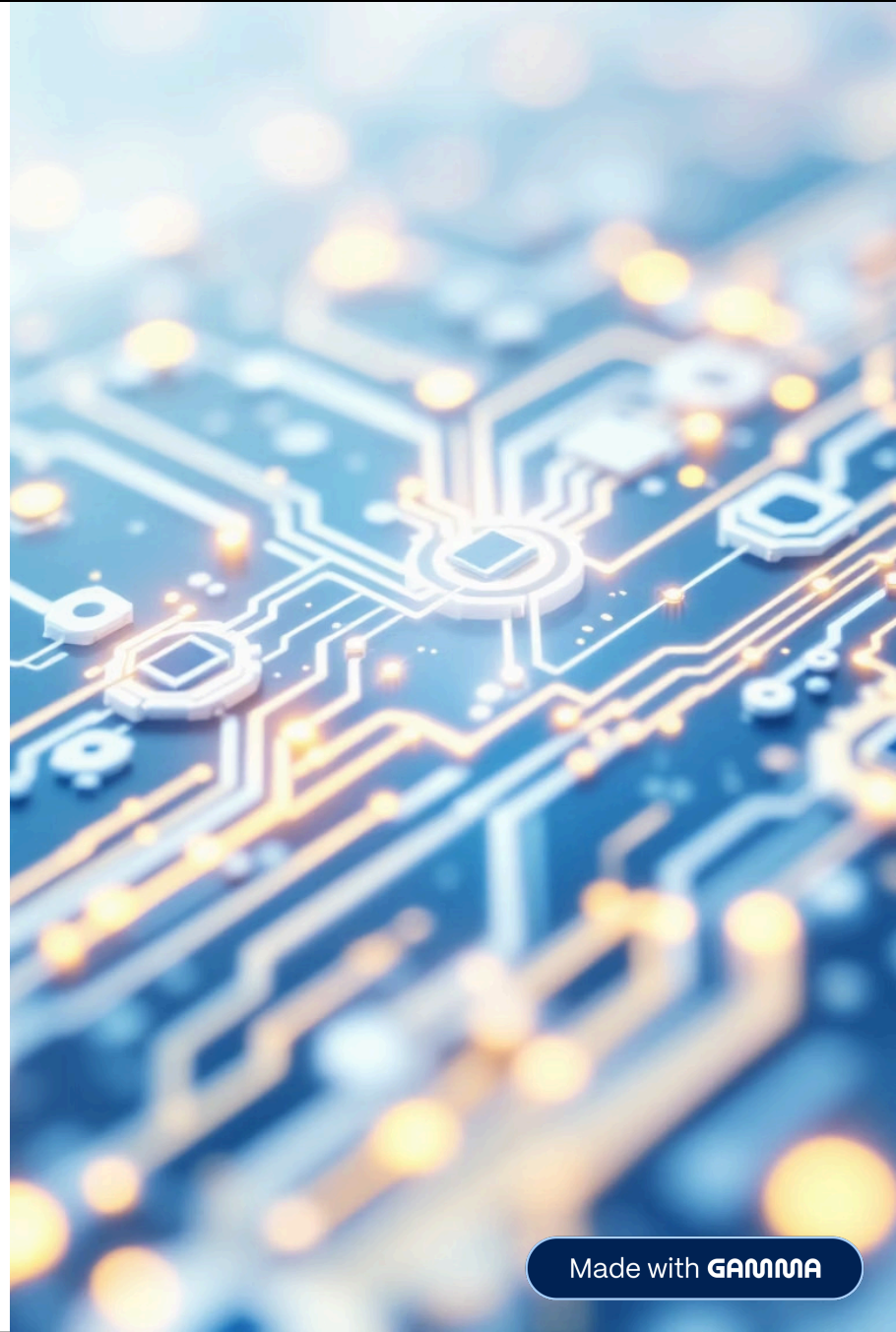
Nhiệm vụ và thành viên

Lê Hà Chức - Lê Quân Đạt

Phát triển và tinh chỉnh RNN LSTM

Lê Thanh Trường

Triển khai workflow MediaPipe–LSTM realtime



Made with GAMMA

Nội dung thuyết trình

- Data ở đâu, cách lấy data, cách xử lý data
- Kiến trúc và cách thức hoạt động
- Đánh giá
- Vì sao chọn RNN này mà không chọn RNN khác
- So sánh với các RNN khác

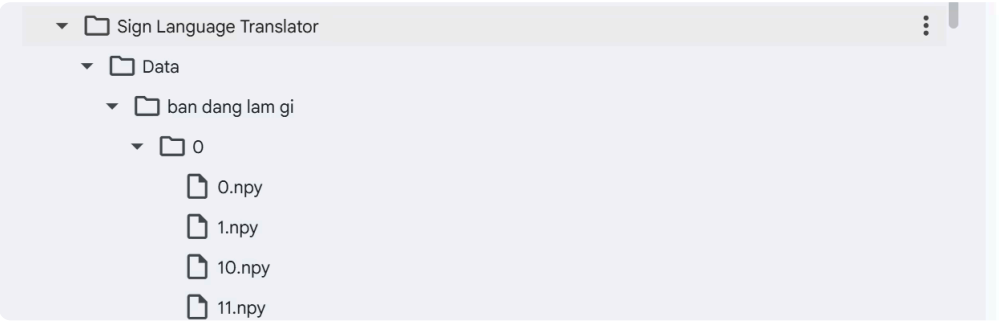
Made with GAMMA

Data ở đâu, cách lấy data, xử lý data

Made with GAMMA

Data ở đâu, cách lấy data

Data được tụi em lấy trên github(https://github.com/khooinguyeen/Sign-Language-Translation) về và tụi em load toàn bộ về máy để sử dụng



Tổng cộng có 60 thư mục hành động.

Bên trong mỗi folder hành động có 60 folder từ 0 - 59 đại diện cho 60 sequences (60 lần quay video cho hành động đó).

Bên trong mỗi folder sequence, có các file `.npy`, đại diện cho các frames (khung hình) của video đó.

Mỗi file `.npy` sẽ chứa 126 features (đặc trưng).

Made with GAMMA

Xử lý data

```python
# Số lượng sequence cho mỗi hành động
NO_SEQUENCES = 60
# Số lượng frames (file .npy) trong mỗi sequence
NO_FRAMES = 60
# Kích thước của đặc trưng
FEATURE_LENGTH = 126
# Lấy danh sách các hành động (tên 60 thư mục)
# Chúng ta sắp xếp (sort) để đảm bảo thứ tự nhãn nhất quán
actions = sorted([d for d in os.listdir(DATA_PATH) if os.path.isdir(os.path.join(DATA_PATH, d))])

# Tạo bộ ánh xạ từ tên hành động sang số (0, 1, 2...)
label_map = {action_name: label_index for label_index, action_name in enumerate(actions)}

print(f"Phát hiện {len(actions)} hành động:")
print(label_map)
```

1

Định nghĩa cấu trúc dữ liệu

NO_SEQUENCES = 60, NO_FRAMES = 60, FEATURE_LENGTH = 126: Đây là các "hằng số" định nghĩa cấu trúc dữ liệu.

2

Lấy tên thư mục hành động

actions = sorted(os.listdir(DATA_PATH)...): Lấy tên của 60 thư mục hành động và đảm bảo thứ tự luôn nhất quán.

3

Mã hóa và gắn nhãn

label_map = {action_name: label_index ...}: Đây là bước **mã hóa** tạo ra một **từ điển** Python

- '0_cam_on' -> 0
- '1_xin_chao' -> 1
- ...
- '59_tam_biet' -> 59

Made with GAMMA

```python
# Khởi tạo các danh sách để lưu dữ liệu
sequences, labels = [], []

# Sử dụng tqdm để xem tiến trình
for action in tqdm(actions, desc="Đang xử lý các hành động"):
    for sequence_index in range(NO_SEQUENCES):
        # Mảng tạm thời để lưu 60 frames của 1 sequence
        window = []

        for frame_index in range(NO_FRAMES):
            file_path = os.path.join(DATA_PATH, action, str(sequence_index), f"{frame_index}.npy")

            try:
                # Tải file .npy
                res = np.load(file_path)
                window.append(res)
            except Exception as e:
                print(f"Lỗi khi đọc file: {file_path} | Lỗi: {e}")
                # Nếu file lỗi, thêm một mảng 0 để giữ đúng shape
                window.append(np.zeros(FEATURE_LENGTH))

        # Thêm chuỗi 60 frames (shape 60, 126) vào danh sách
        sequences.append(window)
        # Thêm nhãn (số) tương ứng
        labels.append(label_map[action])

print(f"\nĐã tải xong {len(sequences)} chuỗi.")
```

1 **Vòng lặp đầu tiên**
sẽ duyệt qua 60 thư mục hành động.

2 **Vòng lặp thứ hai**
sẽ duyệt qua 60 video (sequences) bên trong mỗi thư mục hành động đó.

3 **Vòng lặp thứ ba**
sẽ duyệt qua 60 file `.npy` (frames) của mỗi video.

`try-except` **=>** Nếu 1 file frame bị lỗi/thiếu, chèn một mảng 0 (`np.zeros`) để giữ đúng shape (`60, 126`), tránh làm hỏng toàn bộ dữ liệu.

Made with GAMMA

```python
# Chuyển đổi danh sách Python sang mảng NumPy
# X sẽ có shape (3600, 60, 126)
X_data = np.array(sequences)

# y sẽ có shape (3600,)
y_data = np.array(labels)

print(f"Shape của X_data: {X_data.shape}")
print(f"Shape của y_data: {y_data.shape}")

# Lưu 2 mảng này ra file
output_path_X = "X_data.npy"
output_path_y = "y_data.npy"

np.save(output_path_X, X_data)
np.save(output_path_y, y_data)

print(f"\Đã lưu dữ liệu thành công!")
print(f"File features: {output_path_X}")
print(f"File labels: {output_path_y}")
print(label_map)
```

- `X_data = np.array(sequences)`: Chuyển danh sách `sequences` (gồm 3600 list nhỏ 60x126) thành một khối mảng NumPy 3D

  => `X_data` có shape `(3600, 60, 126)` tức là 3600 video, mỗi video 60 frame, mỗi frame 126 feature.

- `y_data = np.array(labels)`: Chuyển danh sách `labels` (gồm 3600 con số [0, 0,..., 59, 59]) thành một mảng NumPy 1D.

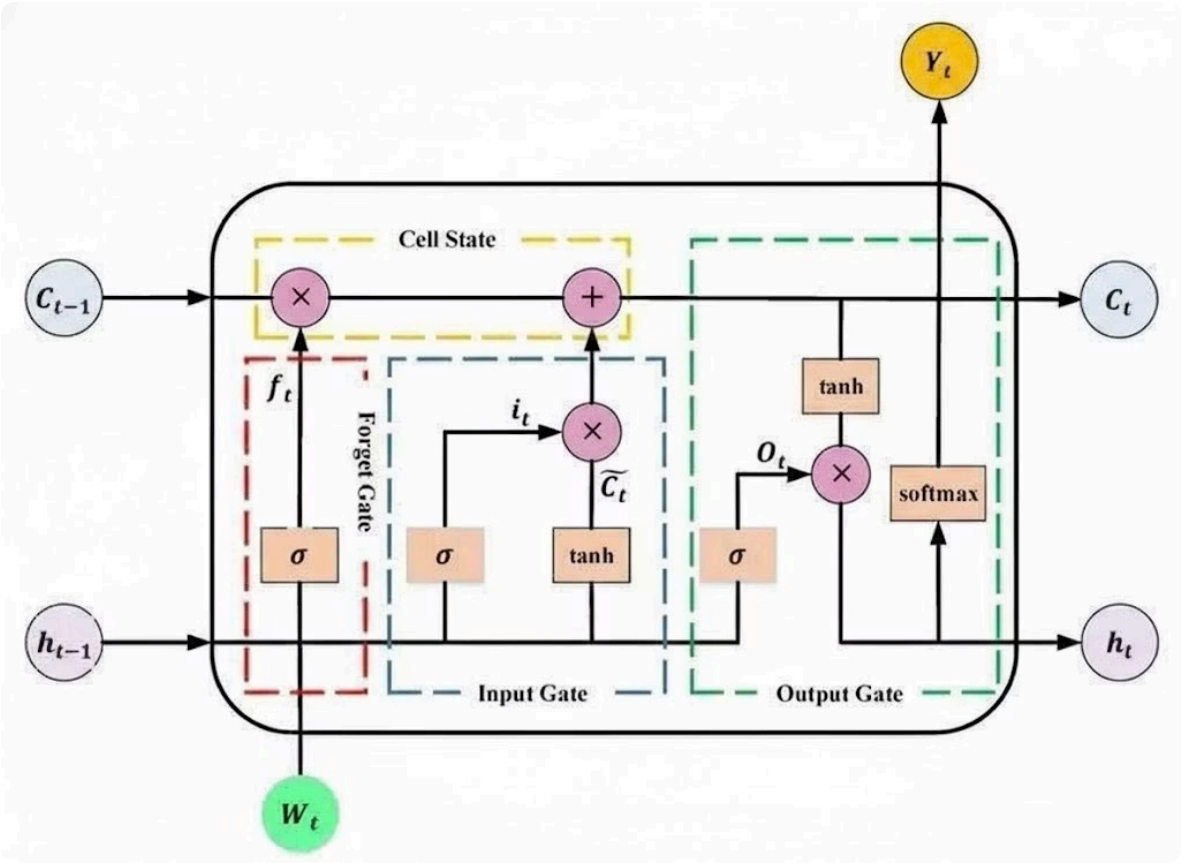
  => `y_data` có shape `(3600,)` tức là 3600 cái nhãn (label) tương ứng cho 3600 video đó.

- `np.save(...)`: Lưu hai mảng này ra file `.npy` để tái sử dụng.

Made with GAMMA

Kiến trúc và cách thức hoạt động

Made with GAMMA



LSTM ra đời năm 1997 nhằm giải quyết vấn đề t**riệt tiêu đạo hàm** (điểm yếu của RNN) của RNN vì không thể học được các mối quan hệ ở xa (long-range dependencies).

- Có thể **ghi nhớ thông tin quan trọng** từ các bước thời gian rất xa và sử dụng nó để đưa ra quyết định ở các bước sau.
- LSTM sử dụng ba cổng để **kiểm soát chặt chẽ** luồng thông tin vào và ra khỏi trạng thái bộ nhớ.
- Có **trạng thái ô giúp** dữ liệu cần thiết truyền qua chuỗi thời gian gần như **không bị thay đổi**, giữ cho gradient không bị triệt tiêu khi lan truyền ngược.

Made with GAMMA

Trạng thái Ô (Cell State)

- **Chức năng:** Bộ nhớ dài hạn
  - Đây là dây thần kinh chủ đạo chạy xuyên suốt từ đầu đến cuối chuỗi (từ frame 1 đến frame 60).
  - Nó lưu trữ thông tin thô và **rất ít khi bị biến đổi**. Thông tin chỉ chạy trượt trên nó và chỉ bị thay đổi khi các Cổng cho phép (thêm vào hoặc xóa bớt).

Vai trò trong bài toán VSL: Giúp mô hình nhớ rằng ở Frame 5 tay đang ở vị trí A, và giữ thông tin đó mãi đến Frame 60 để kết hợp xử lý.

Trạng thái Ẩn (Hidden State)

- **Chức năng:** Bộ nhớ ngắn hạn
  - Đây là bản tóm tắt đã được "lọc" ra từ Cell State.
  - Nó được dùng để đưa ra dự đoán cho thời điểm hiện tại và chuyển sang bước tiếp theo.

Vai trò trong bài toán VSL: Tại Frame 30, Hidden State cho biết "tay đang di chuyển lên", nhưng nó không cần chứa tất cả thông tin chi tiết của quá khứ như Cell State.

Made with GAMMA

Cổng Quên (Forget Gate)

- Câu hỏi: "Thông tin nào từ quá khứ không còn cần thiết nữa?"
- Chức năng: Nó nhìn vào dữ liệu mới và bộ nhớ cũ , sau đó đưa ra quyết định (từ 0 đến 1) cho từng mảnh thông tin trong Cell State.
  - `0`: Xóa ngay lập tức (Quên).
  - `1`: Giữ lại hoàn toàn.

Ví dụ VSL: Khi chuyển từ động tác "giơ tay" sang "nắm tay", Cổng Quên sẽ xóa thông tin "ngón tay đang duỗi" vì nó không còn đúng nữa.

Cổng Nhập (Input Gate)

- Câu hỏi: "Thông tin mới nào quan trọng cần được ghi nhớ?"
- Chức năng: Nó quyết định cập nhật giá trị mới nào vào Cell State. Quy trình gồm các bước:
  - Lọc xem thông tin nào đáng giá (dùng hàm Sigmoid).
  - Tạo ra một vector thông tin mới (dùng hàm Tanh).
  - Cộng thông tin này vào Cell State.

Ví dụ VSL: Camera bắt được chuyển động ngón cái gập xuống. Cổng Nhập thấy quan trọng và ghi vào bộ nhớ: "Lưu ý: Ngón cái đang gập".

Cổng Xuất (Output Gate)

- Câu hỏi: "Dựa trên bộ nhớ hiện tại, tôi nên nói gì với... (hoặc lớp tiếp theo)?"
- Chức năng: Nó quyết định phần nào của Cell State sẽ được dùng để tạo ra Hidden State.
- Nó không xuất toàn bộ bộ nhớ (vì quá nhiều), mà chỉ xuất những gì liên quan đến thời điểm hiện tại.

Ví dụ VSL: Mặc dù bộ nhớ chứa cả quy trình "đưa tay lên -> nắm lại -> hạ xuống", nhưng ở thời điểm hiện tại, Cổng Xuất chỉ báo cáo: "Đang thực hiện hành động hạ tay".

Made with GAMMA

```python
# === 4. Build model ===
model = Sequential([
BatchNormalization(input_shape=(60, 126)),
LSTM(64, return_sequences=True, dropout=0.2, recurrent_dropout=0.2),
LSTM(128, return_sequences=True, dropout=0.3, recurrent_dropout=0.3),
LSTM(64, return_sequences=False, dropout=0.3, recurrent_dropout=0.3),
Dense(128, activation='relu'),
Dropout(0.3),
Dense(60, activation='softmax')
])
```

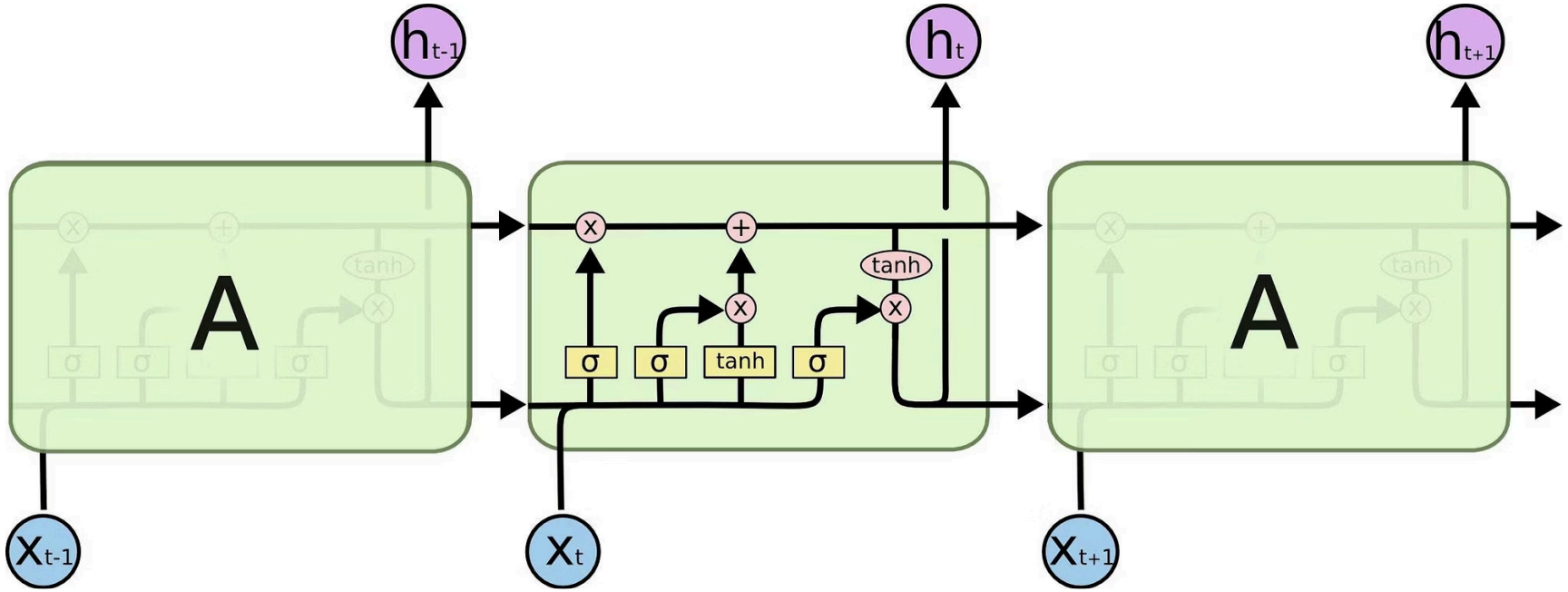
Python

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/normalization/batch_normalization.py:142: UserWar
  super().__init__(**kwargs)
```

BatchNormalization(input_shape=(60, 126))

- **Nhiệm vụ:** Đây là lớp "dọn dẹp" đầu vào. Nó chuẩn hóa 126 features để chúng có cùng một thang đo (ví dụ: đều nằm quanh 0). Giúp mô hình hội tụ nhanh hơn và ổn định hơn.
- *Đầu vào:* `(Batch, 60, 126)`
- *Đầu ra:* `(Batch, 60, 126)` (đã được chuẩn hóa)

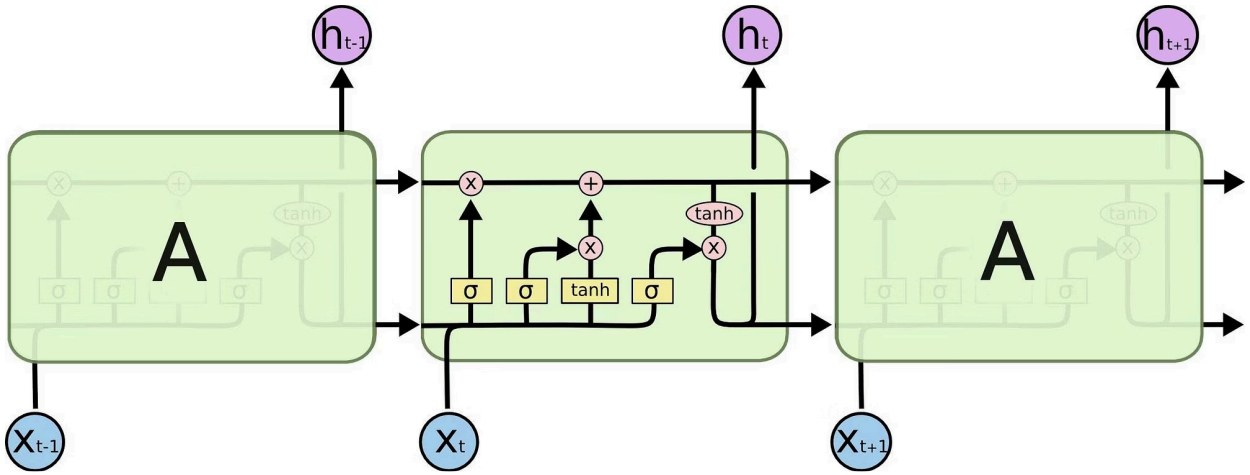
Made with GAMMA



LSTM(64, return_sequences=True)

- **Nhiệm vụ:** Lớp LSTM đầu tiên, học các mẫu (pattern) cơ bản.
- `64` **(units):** Đây là kích thước của "bộ nhớ" (cell state) và "đầu ra" (hidden state) của nó. Nó sẽ "đọc" 126 features ở mỗi timestep và tóm tắt chúng thành 64 features trừu tượng hơn (ví dụ: 126 con số tọa độ giờ được tóm tắt thành 64 con số thể hiện "bàn tay đang mở").
- `return_sequences=True` **(Quan trọng):** Lệnh này bắt buộc lớp LSTM phải trả về kết quả tóm tắt cho *tất cả 60 timestep*.
- **Output Shape:** `(60, 64)`

Made with GAMMA



LSTM(128, return_sequences=True)

- **Nhiệm vụ:** Lớp LSTM thứ hai, học các mẫu phức tạp hơn (mẫu của các mẫu).
- **Cách hoạt động:** Nó nhận đầu vào là chuỗi 60 timestep (mỗi timestep 64 feature). Nó "đọc" chuỗi này và tìm các mẫu cấp cao hơn (ví dụ: Lớp 1 thấy "tay mở", "tay di chuyển lên", Lớp 2 sẽ kết luận "đây là hành động vẫy").
- `128` **(units):** Chúng ta tăng số units để nó có khả năng học các mối quan hệ phức tạp hơn.
- `return_sequences=True`**:** Tương tự, nó trả về *toàn bộ chuỗi* 60 timestep (giờ là 128 feature) cho lớp LSTM cuối cùng.
- **Output Shape:** `(60, 128)`

LSTM(64, return_sequences=False)

- **Nhiệm vụ:** Lớp LSTM cuối cùng, "tổng kết" toàn bộ chuỗi.
- `return_sequences=False` **(Siêu quan trọng):** Đây là điểm khác biệt. Nó vẫn "đọc" toàn bộ 60 timestep đầu vào, nhưng nó *chỉ* trả về đầu ra (hidden state) của **timestep cuối cùng (thứ 60)**.
- **Tại sao** `False`**?** Bởi vì đến timestep 60, bộ nhớ của LSTM này đã chứa đựng thông tin của toàn bộ chuỗi. Vector 64-chiều này chính là "vector bối cảnh" (context vector) – một bản tóm tắt duy nhất đại diện cho *toàn bộ* 60 khung hình.
- **Output Shape:** `(64)` (Không còn 60 timestep nữa, chỉ còn 1 vector 64-chiều).

Made with GAMMA

```
Dense(128, activation='relu'),
Dropout(0.3),
Dense(60, activation='softmax')
])
```

`Dense(128, activation='relu')` & `Dropout(0.3)`

- **Nhiệm vụ:** Đây là lớp **chuyển đổi phi tuyến và mở rộng không gian đặc trưng**. Nó nhận **vector tóm tắt** từ lớp LSTM cuối cùng và biến đổi nó thành một biểu diễn có chiều sâu hơn (128 chiều).
- **Đầu vào:** Vector đặc trưng từ lớp LSTM trước.
- **Đầu ra:** Một vector **128 chiều** đã được kích hoạt bằng hàm **ReLU**.
- `Dropout(0.3)`**:** Kỹ thuật **chống học vẹt (Overfitting) tắt 30%** các nơ-ron đầu ra của lớp `Dense(128)` buộc mô hình không phụ thuộc vào một vài nơ-ron cụ thể, giúp tăng khả năng tổng quát hóa cho dữ liệu mới.

Made with GAMMA

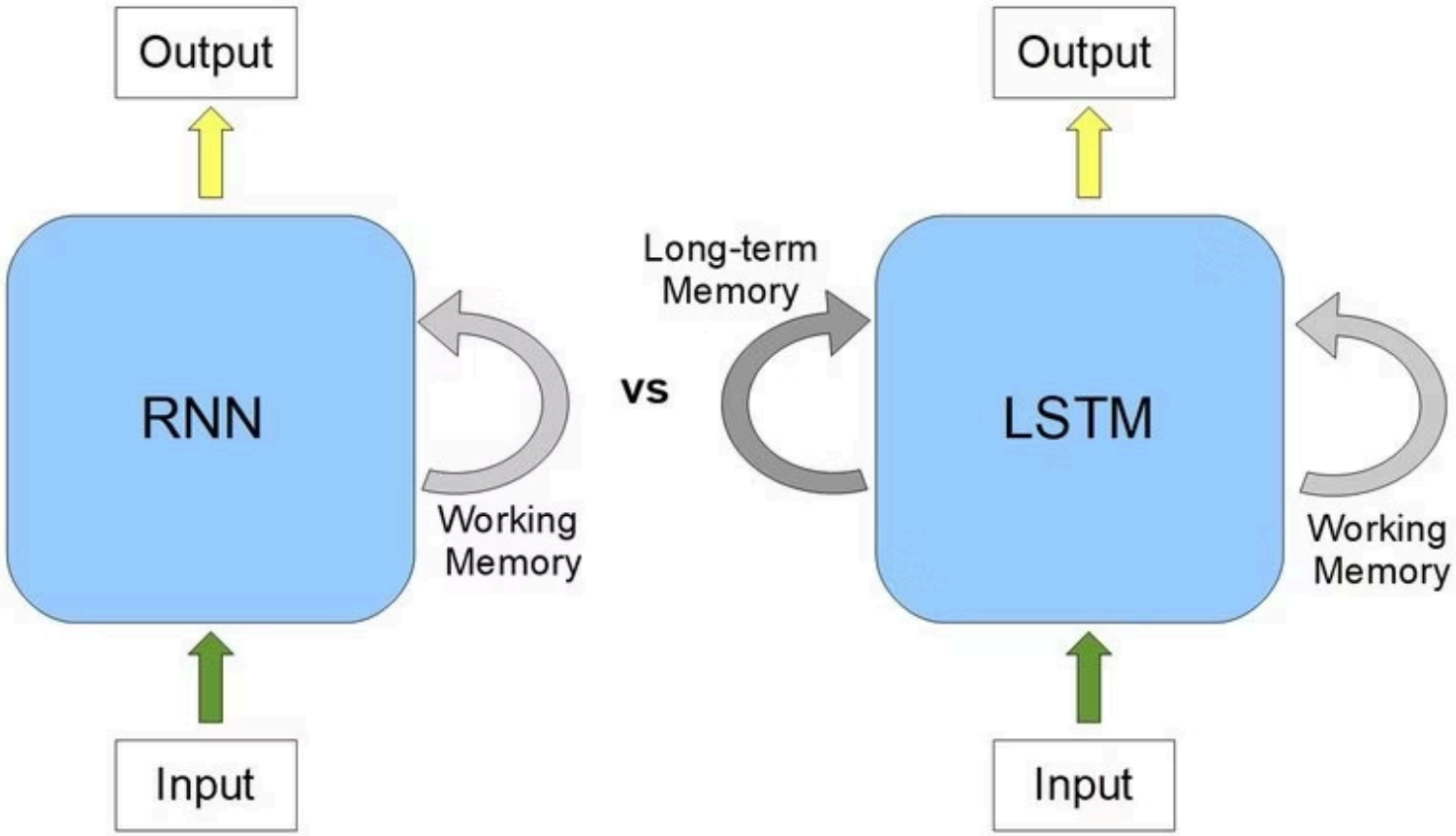
$$\text{Ma trận Xác suất} = \begin{pmatrix} P_{\text{video 1}} \\ P_{\text{video 2}} \\ \vdots \\ P_{\text{video 32}} \end{pmatrix} = \begin{pmatrix} 0.01 & 0.05 & \mathbf{0.90} & \dots & 0.00 \\ 0.03 & \mathbf{0.85} & 0.01 & \dots & 0.02 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{0.70} & 0.10 & 0.05 & \dots & 0.00 \end{pmatrix}$$

Dense(60, activation='softmax')

- **Nhiệm vụ:** Đây là lớp **dự đoán phân loại cuối cùng**. Nó đưa ra quyết định về ký hiệu được nhận diện.
- **Tham số: 60 đơn vị**, tương ứng chính xác với **60 lớp ký hiệu** mà mô hình cần phân loại.
- **Kích hoạt:** Sử dụng hàm softmax.
  - **Công dụng của** softmax**:** Nó biến đổi 60 giá trị đầu ra thành một **vector xác suất**, trong đó tổng xác suất của 60 lớp luôn bằng **1**.
  - *Ví dụ:* [0.01, 0.05, 0.9, ..., 0.04]. Vị trí có xác suất cao nhất (0.9) chính là dự đoán cuối cùng của mô hình.
- **Đầu ra:** Vector xác suất **60 chiều**.

Made with GAMMA

🌟 Vì sao chọn LSTM thay vì RNN thường?



Made with GAMMA

Hiệu năng

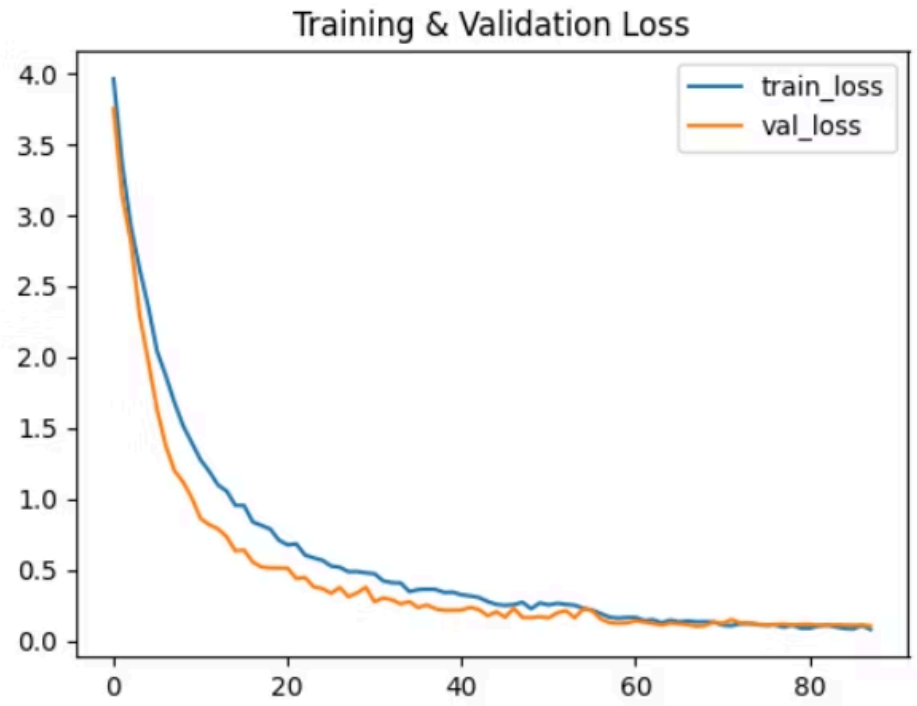
```
23/23 ━━━━━━━━━━━━━━━━━━━━ 4s 168ms/step - categorical_accuracy: 0.9702 - loss: 0.1468
Validation loss: 0.1001 | Validation acc: 0.9736
```

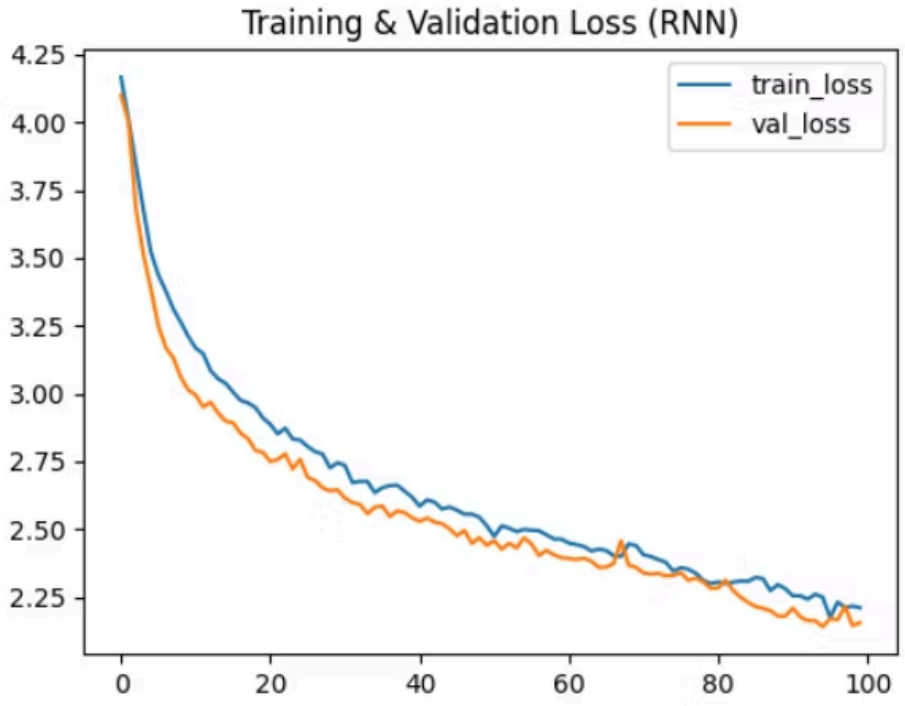
```
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - categorical_accuracy: 0.4012 - loss: 2.1894
Validation loss: 2.1397 | Validation acc: 0.4069
```

- **LSTM:** val_acc ≈ 0.9736, val_loss ≈ 0.1001. Báo cáo phân loại cho thấy đa số lớp đạt **precision/recall ~ 1.00**; nhóm lỗi chính ở các lớp **15, 21, 37, 49** (recall 0.83–0.92).
- **RNN:** val_acc ≈ 0.4069, val_loss ≈ 2.1397; trong báo cáo phân loại có **nhiều lớp bị 0.00** (cả precision/recall/f1), ví dụ lớp **3, 14, 30, 49, 51**; một số lớp riêng lẻ tốt (vd **2, 22, 26, 36, 48** ~ f1 khoảng 0.83–0.86), nhưng **không đồng đều**

Made with GAMMA

Tính ổn định



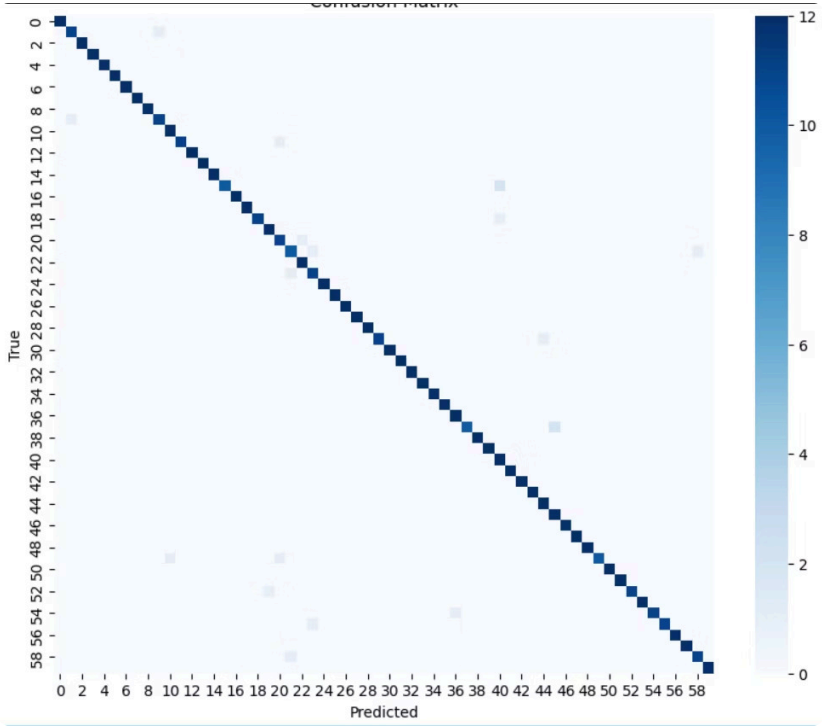


- **LSTM** giảm **val_loss** đều qua các epoch đầu (3.75 → 3.13 → 2.81 → … → 1.00 → 0.86 → …), **không sụt** đột ngột; sau khi giảm LR (ReduceLROnPlateau) vẫn cải thiện → **hội tụ chắc chắn**.

- **RNN** cải thiện chậm (val_loss quanh **4.10 → 3.01 → 2.99**…), **dừng ở loss cao**, dù số tham số ít hơn, mô hình **không học được phụ thuộc dài**, dẫn tới **plateau** sớm.

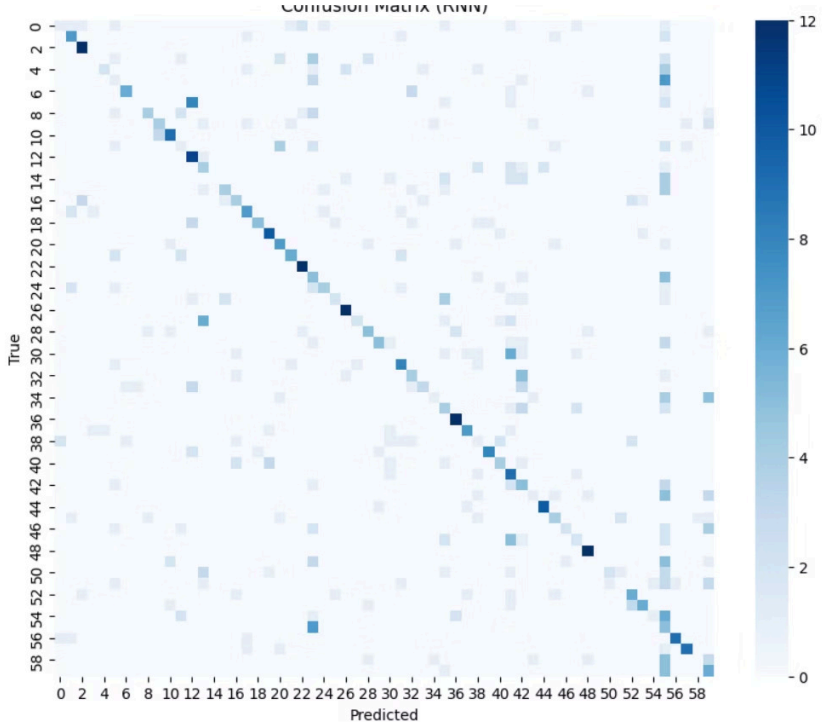
Made with GAMMA

Phân tích nhầm lẫn



LSTM

Ma trận nhầm lẫn **sáng chéo chính**, chỉ vài "vệt" nhầm lẫn nhẹ ở các lớp nêu trên → mô hình **phân ranh lớp tốt**.



RNN thường

Ma trận nhầm lẫn cho thấy **nhiều hàng có phân bố rải, thậm chí dồn vào 1–2 lớp sai**, và **hàng bằng 0** (không dự đoán trúng) ở một số lớp → dấu hiệu **mất nhớ dài hạn/vanishing gradient**.

Made with GAMMA

Bảng so sánh tổng thể

Tiêu chí	RNN thường (SimpleRNN)	LSTM
Kiến trúc khối tuần hoàn	Tích lũy trạng thái đơn giản, không có cổng	Có cổng vào/ghi/ra và cell state (bộ nhớ dài hạn)
Số lớp tuần hoàn	64 → 128 → 64	64 → 128 → 64
Tham số (approx)	65,844	213,684
Validation Accuracy	~0.4069	~0.9736
Validation Loss	~2.1397	~0.1001
Ổn định huấn luyện	Dễ mắc kẹt/plateau, nhiều lớp dự đoán sai toàn bộ	Hội tụ nhanh, ổn định, tránh sụt giảm sau khi giảm LR
Lỗi theo lớp	Nhiều lớp recall = 0 (ví dụ lớp 3, 14, 30, 49, 51)	Lỗi chủ yếu ở vài lớp recall 0.83–0.92 (vd: 15, 21, 37, 49)
Tổng quan	Underfit/khó học phụ thuộc dài	Giữ ngữ cảnh dài, phân biệt lớp tốt

Made with GAMMA

Lý do chọn LSTM thay vì chọn RNN thường

Ghi nhớ dài hạn nhờ cell state + cổng nhớ

Ổn định tối ưu hóa

Chất lượng dự đoán theo lớp

Long-Term Memory

- Cell State
- Memory Gates
- Forget Gate
- Input Gate

Made with GAMMA

Đề xuất cải tiến

- **Data augmentation** (Noise / Time Warping / Mixup) → tăng đa dạng dữ liệu.
- **Label smoothing (ε ≈ 0.05)** → giảm dự đoán quá tự tin, cải thiện Precision.
- **BiLSTM hoặc LayerNorm LSTM** → tăng khả năng nắm ngữ cảnh hai chiều.
- **Dropout ở Dense = 0.4–0.5** nếu có dấu hiệu overfit.

Made with GAMMA

Made with GAMMA

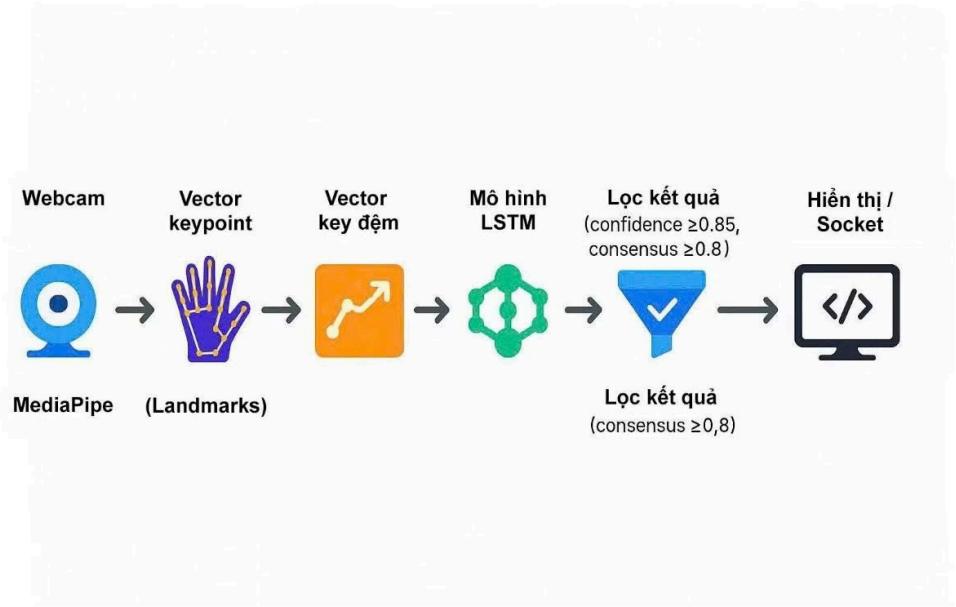
Các Chế Độ Hoạt Động

Chế Độ Real-time

Ở chế độ này, ứng dụng client sẽ liên tục gửi dữ liệu keypoint của người dùng (từ camera) qua giao thức SocketIO. Mô hình AI của chúng ta sẽ xử lý dòng dữ liệu này theo thời gian thực, cung cấp phản hồi tức thì về các ký hiệu được nhận diện. Chế độ này lý tưởng cho việc học tập, thực hành và giao tiếp trực tiếp.

Made with GAMMA

System Workflow Pipeline



- Hệ thống realtime lấy **landmarks tay** từ MediaPipe, chuẩn hoá thành **vector 126 chiều** mỗi frame và xếp vào **cửa sổ 60 frames**.
- Khối **LSTM** dự đoán xác suất 60 lớp; chỉ khi **p ≥ 0.85** và **≥80%** dự đoán gần nhất đồng thuận, nhãn mới được chốt và gửi qua **SocketIO** để hiển thị. Bộ **máy trạng thái IDLE → COUNTDOWN → COLLECTING** giúp người dùng vào tư thế và giảm nhiễu, cho kết quả ổn định và độ trễ thấp.

Made with GAMMA

1. Thu nhận dữ liệu – Webcam

Hệ thống khởi động bằng cách thu nhận luồng hình ảnh trực tiếp từ webcam, sử dụng `cv2.VideoCapture(0)` để truy cập camera mặc định.

- Webcam hoạt động ở tốc độ **khoảng 30 khung hình/giây (FPS)**.
- Mỗi khung hình có độ phân giải **640×480 pixel**.
- Mỗi khung hình sau khi được ghi lại sẽ **ngay lập tức được gửi tới MediaPipe** để tiến hành phân tích và trích xuất đặc trưng.

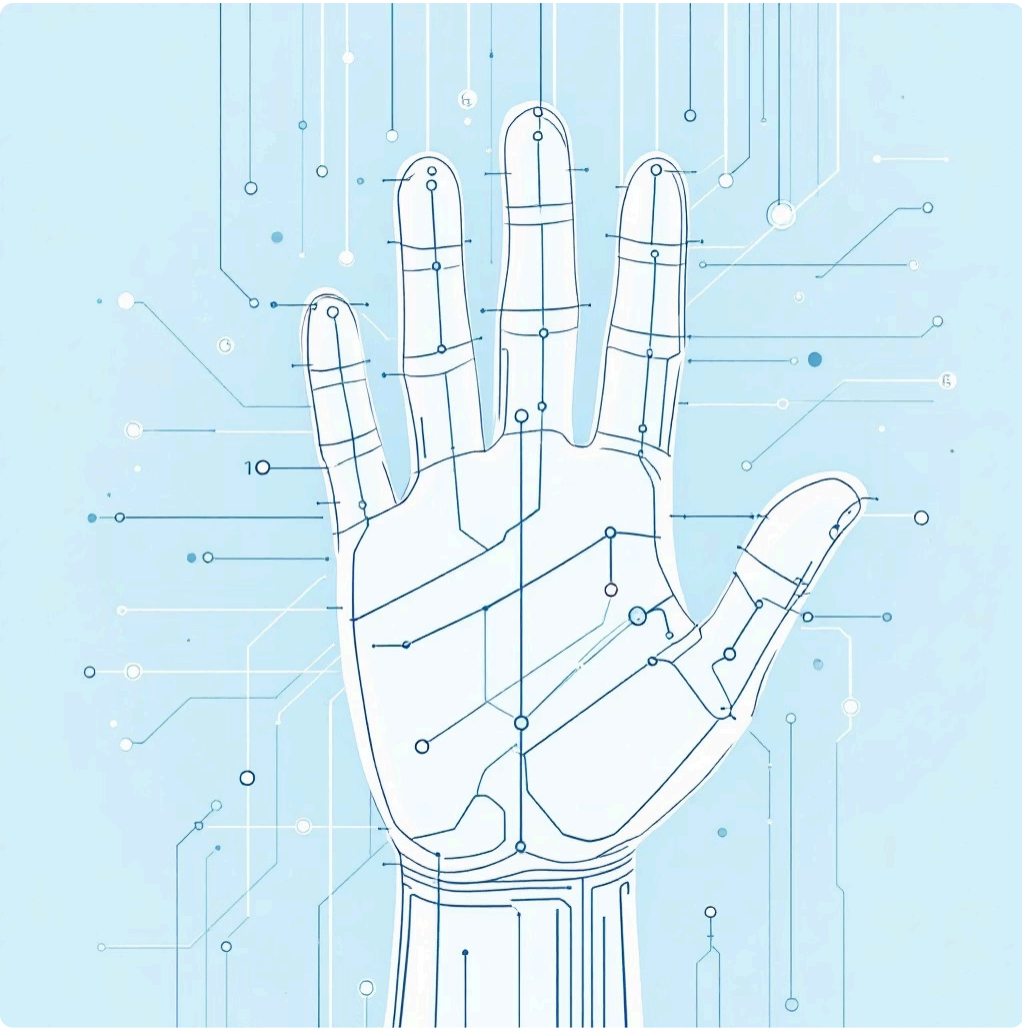
Made with GAMMA

2. Trích xuất đặc trưng – MediaPipe

Sau khi có được hình ảnh từ webcam, bước tiếp theo là trích xuất đặc trưng bàn tay bằng MediaPipe.

Sử dụng **MediaPipe Holistic** để phát hiện 21 điểm `(x,y,z)` trên mỗi bàn tay.

- Các điểm này bao gồm vị trí của khớp ngón tay, đầu ngón tay và cổ tay.
- Nếu không phát hiện được bàn tay, MediaPipe sẽ trả về một vector toàn số 0, giúp tránh lỗi cho mô hình.

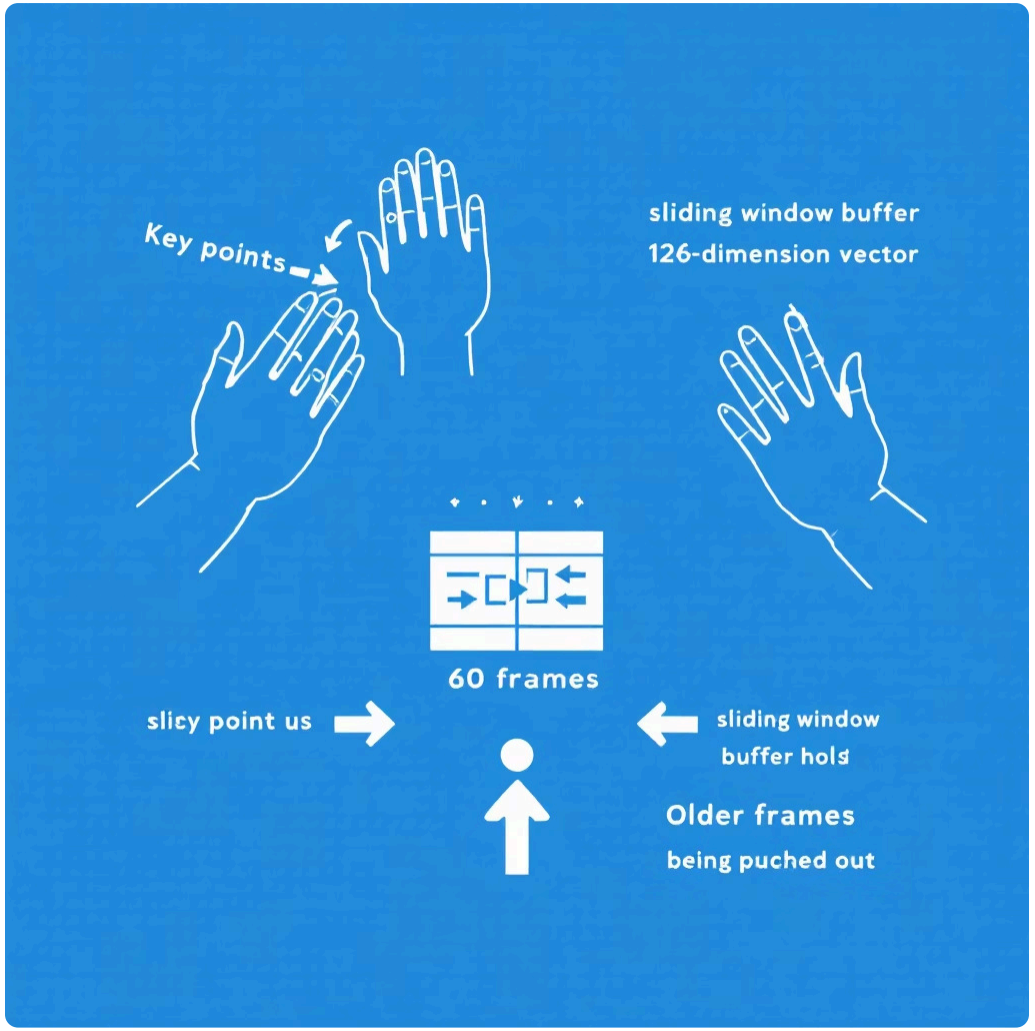


Made with GAMMA

3. Vector hóa & Bộ đệm keypoint

Sau khi trích xuất landmarks, chúng em chuyển sang bước chuẩn hóa để mô hình có thể hiểu được dữ liệu.

- Hàm `extract_keypoints()` sẽ nối 2 bàn tay, tạo thành một vector **126 giá trị** (21 điểm × 3 tọa độ × 2 tay).
- Hệ thống duy trì một bộ đệm sequence chứa **60 frame** keypoint gần nhất, tương ứng với khoảng **2 giây** dữ liệu chuyển động.
- Với mỗi frame mới được thu nhận, frame cũ nhất trong bộ đệm sẽ bị loại bỏ, tạo thành một **chuỗi thời gian trượt** (sliding window), đảm bảo mô hình luôn phân tích ngữ cảnh chuyển động gần đây nhất.



Made with GAMMA

4. Máy trạng thái điều khiển (State Machine)

Để hệ thống hoạt động mượt và tránh nhận nhầm khi người dùng chưa sẵn sàng, chúng em xây dựng một state machine.

IDLE

Không thấy tay, chờ người dùng đưa tay vào.

COUNTDOWN

Khi thấy tay, hệ thống đếm ngược 3 giây để người dùng chuẩn bị.

COLLECTING

Bắt đầu thu 60 frame và dự đoán liên tục.

Lưu ý: Nếu hệ thống mất tín hiệu tay người dùng trong hơn 2 giây, nó sẽ tự động reset về trạng thái `IDLE`.

Made with GAMMA

5. Mô hình LSTM – Nhận diện ký hiệu

Đây là phần quan trọng nhất: mô hình học sâu chịu trách nhiệm nhận diện ký hiệu, được huấn luyện để phân tích các chuỗi dữ liệu keypoint và dự đoán ký hiệu ngôn ngữ ký hiệu tiếng Việt.

- Mô hình được huấn luyện trước: `vsl_lstm_model_v3.h5`.
- Dạng Input: `(1, 60, 126)` – tương ứng 60 khung hình, mỗi khung hình 126 giá trị keypoint.
- Cấu trúc bên trong:
  - Các lớp LSTM để học chuỗi thời gian.
  - Các lớp Dense để xử lý và trích xuất đặc trưng.
  - Lớp Softmax với 60 kết quả tương ứng 60 hành động/ký hiệu.

Made with GAMMA

6. Lọc kết quả & Xây dựng câu

Dự đoán thô từ mô hình còn khá nhiễu, nên chúng em áp dụng thêm bước lọc thông minh.

Độ tự tin cao

Hệ thống chỉ chấp nhận một từ khi độ tự tin của mô hình phải từ **0.85** trở lên.

Độ đồng thuận cao

Ít nhất **80%** (24/30 frame) các dự đoán gần nhất phải trùng khớp nhau.

Điều này giúp:

→ Tránh **rung dự đoán**: Đảm bảo kết quả ổn định, không bị "nhảy" liên tục khi có thay đổi nhỏ.

→ Tránh **nhầm lẫn**: Ngăn chặn việc nhận diện sai khi người dùng đổi tư thế hoặc có cử động thừa.

Kết quả được lọc sẽ được thêm vào câu và chỉ hiển thị tối đa 5 từ gần nhất để đảm bảo sự liền mạch.

Made with GAMMA

7. Hiển thị kết quả

Kết quả nhận dạng

Hiển thị ký hiệu đang được mô hình nhận diện trong thời gian thực, giúp người dùng theo dõi và điều chỉnh kịp thời.

Câu hoàn chỉnh

Các từ được nhận diện chính xác sẽ được thêm vào, tạo thành một câu hoàn chỉnh, giúp người dùng dễ dàng theo dõi dòng chảy giao tiếp.

Countdown chuẩn bị

Hiển thị đồng hồ đếm ngược trước khi bắt đầu nhận diện, đảm bảo người dùng có đủ thời gian chuẩn bị tư thế.

Giao diện trực quan

Tích hợp các viền và overlay đồ họa thân thiện, làm nổi bật thông tin quan trọng và nâng cao trải nghiệm người dùng trên màn hình webcam.

Made with GAMMA

Đánh giá hiệu suất

Mục tiêu của quá trình đánh giá là chứng minh hệ thống hoạt động ổn định và hiệu quả trong môi trường thực tế.

Chỉ số hiệu suất

- **Độ chính xác trung bình:** ~85–90%
- **Độ trễ:** ~2 giây (tương đương 60 khung hình xử lý)

Điểm mạnh

- **Thời gian thực:** Xử lý và phản hồi tức thì.
- **Tiện lợi:** Không cần thiết bị đeo như găng tay cảm biến.

Điểm hạn chế

- **Ánh sáng:** Hiệu suất có thể bị ảnh hưởng bởi điều kiện ánh sáng kém.
- **Góc quay:** Độ chính xác phụ thuộc vào góc quay của camera.

Made with GAMMA

Demo web Việt Nam Sign Language

Made with GAMMA